UNIVERSIDAD PERUANA DE CIENCIAS APLICADAS

Facultad de Ingeniería

Carrera de Ciencias de la Computación

Teoría de Compiladores (1ACC0218)

Trabajo Final (TB2) – Hito 2: segundo avance

HospSim: un lenguaje para describir y simular la atención de pacientes en un hospital

Sección: 4889

Profesor: José Luis Soncco Álvarez

Integrantes

Rivera Calderón, Alvaro Jerry (u20231h075)
Alarcon Castellanos, Ericks Santiago (U202322704)
Colonio Soto, Rodrigo Aaron (u202323598)

Repositorio: https://github.com/Jere348/HospSim

Lima, noviembre de 2026

Índice

1. Problemática y motivación.....
2. Objetivos.....
2.1 Objetivo general.....
2.2 Objetivos específicos.....
3. Construcciones del lenguaje.....
3.1 Declaración de recursos.....
3.2 Declaración de procesos.....
3.3 Declaración de etapas.....
3.4 Generación de llegadas.....
3.5 Simulación.....
3.6 Consulta de métricas.....
3.7 Expresiones aritméticas.....
3.8 Un programa completo.....
4. Analizador léxico.....
4.1 Léxico del lenguaje.....
4.2 Implementación en ANTLR4.....
4.3 Pruebas.....
5. Analizador sintáctico.....
5.1 Gramática.....
5.2 Gramática por construcción.....
5.3 Qué cambió desde el Hito 1.....
5.4 Una derivación de ejemplo.....
5.5 Implementación y pruebas.....
6. Analizador semántico.....
6.1 Tabla de símbolos.....
6.2 Errores que detecta.....
7. Arquitectura del compilador.....
7.1 Decisiones de diseño.....
7.2 Código intermedio.....
8. Plan de validación.....
8.1 Método.....
8.2 Casos.....
8.3 Resultados calculados a mano.....
8.4 Resultados de la ejecución.....
9. Estado del avance.....
Referencias.....

Anexo A. Derivaciones más a la izquierda.....
A.1 Programa.....
A.2 Declaración de recurso.....
A.3 Declaración de proceso con sus etapas.....
A.4 Generación de llegadas.....
A.5 Simulación.....
A.6 Consulta de métricas.....
A.7 Expresiones aritméticas.....
Anexo B. Bitácora del uso de IA.....

1. Problemática y motivación

Supongamos un consultorio con un solo médico. Cada consulta dura 15 minutos y llega un paciente cada 5. Al cabo de una hora el médico habrá atendido a cuatro personas, la quinta recién estará entrando y cada paciente nuevo espera diez minutos más que el anterior. Con un médico y una sola etapa la cuenta se hace en una hoja de papel. Con triaje, dos médicos, camas que comparten emergencia y hospitalización y pacientes que llegan a ritmos distintos, ya no.

En la práctica, cuántos médicos poner en un turno o cuántas camas reservar se decide por experiencia o con una hoja de cálculo. La hoja de cálculo maneja bien los promedios, pero no muestra cómo crece una cola cuando dos pacientes piden el mismo recurso al mismo tiempo. Para eso existe la simulación de eventos discretos (Banks et al., 2010).

Herramientas de simulación hay. El problema que vimos es lo que piden a cambio: una licencia comercial con su capacitación, o saber programar, como en el caso de SimPy, una biblioteca de Python (Team SimPy, s. f.). Quien mejor conoce el flujo de pacientes, la jefa de un servicio de emergencia por ejemplo, casi nunca cumple la segunda condición, y armar un modelo termina siendo un ida y vuelta largo entre ella y un programador.

HospSim es un lenguaje pequeño, en español, cuyas palabras reservadas vienen del propio dominio: *recurso*, *proceso*, *etapa*, *pacientes*. El Listado 1 describe el consultorio del primer párrafo. Antes de simular, el compilador revisa el modelo y avisa, por ejemplo, si una etapa usa un recurso que nadie declaró o si una capacidad no es un entero positivo. Después ejecuta la simulación y muestra las métricas que el programa pidió.

Listado 1

Consultorio con un médico escrito en HospSim y salida del compilador

```
recurso medicos = 1;
proceso atencion {
    etapa consulta usa medicos durante 15 minutos;
}
generar pacientes cada 5 minutos hacia atencion;
simular durante 1 hora;
mostrar pacientes_atendidos;
mostrar espera_promedio;
mostrar ocupacion(medicos);

Análisis semántico: OK
=== RESULTADOS ===
  pacientes_atendidos = 4
  espera_promedio = 15.00 minutos
  ocupacion(medicos) = 100.00 %
```

Nota. Archivo `ejemplos/atencion.txt`. Las tres últimas líneas son la salida de `python main.py ejemplos/atencion.txt`.

2. Objetivos

2.1 Objetivo general

Construir un compilador para HospSim que valide un modelo de atención hospitalaria escrito en el lenguaje, lo traduzca a una representación intermedia y lo ejecute como simulación de eventos discretos.

2.2 Objetivos específicos

- Especificar el léxico de HospSim como pares token y lexemas, e implementarlo en ANTLR4.
- Escribir una gramática libre de contexto no ambigua para cada construcción y generar con ella el analizador sintáctico.
- Detectar en el análisis semántico al menos los seis errores que pide el Hito 2. Hoy el compilador comprueba 22.
- Emitir el código intermedio del modelo validado y ejecutarlo con un simulador determinista.
- Probar cada fase con casos automáticos cuyas salidas esperadas calculamos a mano antes de aceptarlas.

3. Construcciones del lenguaje

Un programa HospSim es una lista de sentencias terminadas en punto y coma. Todo identificador se declara antes de usarse. Hay un solo tipo de dato, el número, entero o decimal, y los tiempos llevan unidad: *minuto*, *minutos*, *hora* u *horas*; el compilador convierte todo a minutos. Los comentarios empiezan con `//` y terminan con la línea.

Distinguimos siete construcciones. Los cinco ejemplos de cada una pasaron por el analizador sintáctico antes de entrar al informe; el script que arma las tablas se detiene si alguno tiene un error de sintaxis.

3.1 Declaración de recursos

Un recurso es todo aquello por lo que un paciente puede tener que esperar: médicos, camas, un ecógrafo. El número a la derecha del igual es la capacidad, o sea, cuántos pacientes atiende a la vez. Puede escribirse como expresión y usar recursos declarados antes.

Tabla 1
Ejemplos: declaración de recursos

N.°	Ejemplo
1	`recurso medicos = 3;`
2	`recurso camas = 20;`
3	`recurso enfermeros = medicos * 2;`
4	`recurso boxes_triaje = (camas - 4) / 4;`
5	`recurso tecnicos_rx = 1;`

Nota. Forma general: `recurso id = expresión ;`

3.2 Declaración de procesos

Un proceso es un recorrido de atención. Sus etapas se cumplen en el orden en que están escritas. Los nombres de etapa pertenecen al ámbito del proceso, así que dos procesos distintos pueden tener cada uno su etapa *consulta*.

Tabla 2
Ejemplos: declaración de procesos

N.°	Ejemplo
1	`proceso consulta_externa { etapa consulta usa medicos durante 20 minutos; }`
2	`proceso emergencia { etapa triaje usa enfermeros durante 5 minutos; etapa atencion usa medicos durante 30 minutos; }`
3	`proceso hospitalizacion { etapa admision usa admisionistas durante 15 minutos; etapa internamiento usa camas durante 48 horas; }`
4	`proceso imagenes { etapa radiografia usa tecnicos_rx durante 10 minutos; etapa informe usa radiologos durante 0.5 horas; }`
5	`proceso cirugia_menor { etapa preparacion usa enfermeros durante 20 minutos; etapa operacion usa cirujanos durante 1 hora; etapa recuperacion usa camas durante 2 horas; }`

Nota. Forma general: `proceso id { etapas }`

3.3 Declaración de etapas

Cada etapa indica qué recurso ocupa el paciente y por cuánto tiempo. Mientras dura, el paciente retiene una unidad de la capacidad de ese recurso.

Tabla 3
Ejemplos: declaración de etapas

N.°	Ejemplo
1	`etapa triaje usa enfermeros durante 5 minutos;`
2	`etapa consulta usa medicos durante 15 minutos;`
3	`etapa internamiento usa camas durante 2 * 24 horas;`
4	`etapa laboratorio usa tecnicos durante 45 minutos;`
5	`etapa alta usa admision durante 1 hora;`

Nota. Forma general: `etapa id usa id durante expresión unidad ;`

3.4 Generación de llegadas

Fija cada cuánto llega un paciente y a qué proceso entra. El primero llega en el minuto 0. Puede haber varias sentencias *generar*, incluso hacia el mismo proceso.

Tabla 4
Ejemplos: generación de llegadas

N.°	Ejemplo
1	`generar pacientes cada 5 minutos hacia emergencia;`
2	`generar pacientes cada 12 minutos hacia consulta_externa;`
3	`generar pacientes cada 1 hora hacia hospitalizacion;`
4	`generar pacientes cada 0.5 horas hacia imagenes;`
5	`generar pacientes cada 60 / 4 minutos hacia emergencia;`

Nota. Forma general: `generar pacientes cada expresión unidad hacia id ;`

3.5 Simulación

Indica cuánto tiempo se simula. Debe haber exactamente una por programa.

Tabla 5
Ejemplos: simulación

N.°	Ejemplo
1	`simular durante 8 horas;`
2	`simular durante 12 horas;`
3	`simular durante 90 minutos;`
4	`simular durante 1 hora;`
5	`simular durante 24 * 7 horas;`

Nota. Forma general: `simular durante expresión unidad ;`

3.6 Consulta de métricas

Pide un resultado al terminar la simulación. *pacientes_atendidos* y *espera_promedio* van solas; *ocupacion* lleva entre paréntesis el recurso que se quiere medir.

Tabla 6
Ejemplos: consulta de métricas

N.°	Ejemplo
1	`mostrar pacientes_atendidos;`
2	`mostrar espera_promedio;`
3	`mostrar ocupacion(medicos);`
4	`mostrar ocupacion(camas);`
5	`mostrar ocupacion(enfermeros);`

Nota. Forma general: `mostrar id ;  |  mostrar id ( id ) ;`

3.7 Expresiones aritméticas

Aparecen en capacidades, duraciones e intervalos. Admiten números, paréntesis, los cuatro operadores y nombres de recursos. Un recurso dentro de una expresión vale su capacidad.

Tabla 7
Ejemplos: expresiones aritméticas

N.°	Ejemplo
1	`15`
2	`2 * 10`
3	`medicos + 4`
4	`(camas - 4) / 2`
5	`24 * 7 - 8`

Nota. Forma general: `E → E + T | E − T | T, con T y F`

3.8 Un programa completo

El Listado 2 es el caso más largo de nuestras pruebas: un triaje con dos médicos y cinco camas durante un turno de ocho horas. En la sección 8.3 explicamos de dónde sale cada número de la salida.

Listado 2
Triaje y hospitalización durante un turno de ocho horas

```
// ejemplo completo
recurso medicos = 2 * 1;
recurso camas = 5;
proceso atencion {
  etapa triaje usa medicos durante 10 minutos;
  etapa cama usa camas durante 1 hora;
}
generar pacientes cada 5 minutos hacia atencion;
simular durante 8 horas;
mostrar pacientes_atendidos;
mostrar ocupacion(medicos);
mostrar ocupacion(camas);
mostrar espera_promedio;

Análisis semántico: OK
=== RESULTADOS ===
  pacientes_atendidos = 35
  ocupacion(medicos) = 99.48 %
  ocupacion(camas) = 95.83 %
  espera_promedio = 105.00 minutos
```

Nota. Archivo `tests/entrada_completa.txt` y su salida.

4. Analizador léxico

4.1 Léxico del lenguaje

HospSim tiene 15 palabras reservadas, 10 signos de operación y puntuación y dos clases de token definidas por patrón, ID y NUMBER (Tabla 8). Las métricas no figuran como palabras

reservadas. Las tratamos como identificadores que ya están declarados en el ámbito global de la tabla de símbolos; si más adelante agregamos una métrica, el analizador léxico no cambia.

Tabla 8
Léxico de HospSim en la forma token: lista de lexemas

Token	Lexemas	Descripción o patrón
`RECURSO, PROCESO, ETAPA`	`recurso, proceso, etapa`	Palabras reservadas de declaración
`USA, DURANTE, HACIA, CADA`	`usa, durante, hacia, cada`	Palabras reservadas de enlace
`GENERAR, PACIENTES`	`generar, pacientes`	Palabras reservadas de llegada
`SIMULAR, MOSTRAR`	`simular, mostrar`	Ejecución y consulta
`MINUTO, MINUTOS`	`minuto, minutos`	Unidad de tiempo (factor 1)
`HORA, HORAS`	`hora, horas`	Unidad de tiempo (factor 60)
`ASSIGN`	`=`	Capacidad de un recurso
`SEMI`	`;`	Fin de sentencia
`LBRACE, RBRACE`	`{, }`	Cuerpo de un proceso
`LPAREN, RPAREN`	`(, )`	Agrupación y argumento de métrica
`PLUS, MINUS`	`+, -`	Operadores aditivos
`MULT, DIV`	`*, /`	Operadores multiplicativos
`ID`	`medicos, camas, consulta_externa, triaje_1, ocupacion`	`[a-zA-Z_][a-zA-Z0-9_]*`
`NUMBER`	`0, 15, 48, 2.5, 0.5`	`[0-9]+ ('.' [0-9]+)?`
`WS (descartado)`	espacio, tabulación, salto de línea	`[ \t\r\n]+`
`COMMENT (descartado)`	`// hasta el fin de la línea`	`'//' ~[\r\n]*`

4.2 Implementación en ANTLR4

En `HospSim.g4` las palabras reservadas se declaran antes que la regla ID. Cuando dos reglas reconocen el mismo lexema con la misma longitud, ANTLR elige la que aparece primero (Parr, 2013), y por eso *proceso* se lee como PROCESO y no como ID. Si las longitudes difieren gana el lexema más largo: *procesos* es un ID y *minutos* no se parte en *minuto* más una *s*. Espacios y comentarios se descartan con `-> skip`.

4.3 Pruebas

Al driver le agregamos la opción `--tokens`, que imprime cada token con su línea y columna. El Listado 3 muestra el comienzo del Listado 1 visto por el analizador léxico.

Listado 3
Primeros tokens del programa del Listado 1

```
1:0    RECURSO    'recurso'
1:8    ID         'medicos'
1:16   ASSIGN     '='
1:18   NUMBER     '1'
1:19   SEMI       ';'
3:0    PROCESO    'proceso'
3:8    ID         'atencion'
3:17   LBRACE     '{'
4:4    ETAPA      'etapa'
4:10   ID         'consulta'
4:19   USA        'usa'
4:23   ID         'medicos'
4:31   DURANTE    'durante'
4:39   NUMBER     '15'
4:42   MINUTOS    'minutos'
4:49   SEMI       ';'
5:0    RBRACE     '}'
...
```

Nota. Salida de `python main.py --tokens tests/entrada_correcta.txt`. Formato: línea:columna, token y lexema.

El carácter `@` no pertenece al lenguaje. El analizador léxico lo reporta con su posición y continúa (Listado 4). En una compilación normal aparece además el error sintáctico que provoca el `2` que queda suelto, y el driver se detiene sin llegar al análisis semántico.

Listado 4
Error léxico en recurso m = 1 @ 2;

```
1:0    RECURSO    'recurso'
1:8    ID         'm'
1:10   ASSIGN     '='
1:12   NUMBER     '1'
1:16   NUMBER     '2'
1:17   SEMI       ';'
  [Error Léxico, línea 1:14] token recognition error at: '@'

$ python main.py tests/error_lexico.txt
=== ERRORES LÉXICOS / SINTÁCTICOS ===
  [Error Léxico, línea 1:14] token recognition error at: '@'
  [Error Sintáctico, línea 1:16] extraneous input '2' expecting ';'
```

5. Analizador sintáctico

5.1 Gramática

Escribimos la gramática con la convención de notación vista en clase (Soncco Álvarez, s. f.; Aho et al., 2007): no terminales en cursiva, terminales en negrita, ε para la cadena vacía y la cabeza de la primera producción, *programa*, como símbolo inicial. *E*, *T* y *F* son expresión, término y factor; **id** y **num** corresponden a los tokens ID y NUMBER. La gramática completa está en la Tabla 9.

Tabla 9
Gramática G de HospSim

programa	→	*sentencias*
sentencias	→	*sentencia sentencias* | ε
sentencia	→	*decl_recurso* | *decl_proceso* | *sent_llegada* | *sent_simular* | *sent_mostrar*
decl_recurso	→	**recurso id** = *E* ;
decl_proceso	→	**proceso id** { *etapas* }
etapas	→	*decl_etapa etapas* | ε
decl_etapa	→	**etapa id usa id durante** *E unidad* ;
sent_llegada	→	**generar pacientes cada** *E unidad* **hacia id** ;
sent_simular	→	**simular durante** *E unidad* ;
sent_mostrar	→	**mostrar id** ; | **mostrar id** (**id**) ;
unidad	→	**minuto** | **minutos** | **hora** | **horas**
E	→	*E* + *T* | *E* − *T* | *T*
T	→	*T* * *F* | *T* / *F* | *F*
F	→	(*E*) | **num** | **id**

Nota. Las repeticiones `X*` de ANTLR aparecen como recursión con ε en *sentencias* y *etapas*.

5.2 Gramática por construcción

El enunciado pide una gramática para cada construcción. No las repetimos por separado porque cada una es la parte de G que se alcanza desde su no terminal. La de una llegada, por ejemplo, contiene *sent_llegada*, *unidad*, *E*, *T* y *F*. El Anexo A trae cuatro derivaciones más a la izquierda para cada una de las siete, 28 en total.

5.3 Qué cambió desde el Hito 1

La gramática del Hito 1 tenía un error que no vimos a tiempo. Las expresiones se definían con una sola regla, `expr : expr (PLUS|MINUS) expr | expr (MULT|DIV) expr | …`, con la suma escrita antes que la multiplicación. Esa regla es ambigua, y ANTLR resuelve la ambigüedad dándole prioridad a la alternativa que aparece primero. Resultado: `recurso camas = 2 + 3 * 4;` declaraba 20 camas, no 14.

Lo descubrimos al preparar este avance, imprimiendo el árbol de esa expresión. La corrección fue pasar a la gramática estratificada que se vio en clase, con *E* para sumas y restas, *T* para productos y cocientes y *F* para números, identificadores y paréntesis (Listado 5). Ahora la precedencia y la asociatividad por la izquierda salen de la gramática misma: 20 − 5 − 3 vale 12 y 24 / 4 / 2 vale 3. Las pruebas *sim_precedencia* y *sim_asociatividad* fallan si alguien vuelve a la versión anterior.

Listado 5
Reglas de expresiones en HospSim.g4

```
expr
    : expr (PLUS | MINUS) term     # AddExpr
    | term                         # TermExpr
    ;

term
    : term (MULT | DIV) factor     # MultExpr
    | factor                       # FactorExpr
    ;

factor
    : NUMBER                       # NumExpr
    | ID                           # IdExpr
    | LPAREN expr RPAREN           # ParenExpr
    ;
```

Nota. Cada etiqueta (`# AddExpr`, `# MultExpr`, …) genera un método *visit* propio en el visitor.

5.4 Una derivación de ejemplo

La derivación más a la izquierda de `2 + 3 * 4` en G es única. En el quinto paso la multiplicación aparece al expandir *T*, un nivel por debajo de la suma, y por eso se agrupa primero.

Entrada: `2 + 3 * 4`

E	
$\Rightarrow_{lm}$ *E* + *T*	E → E + T
$\Rightarrow_{lm}$ *T* + *T*	E → T
$\Rightarrow_{lm}$ *F* + *T*	T → F
$\Rightarrow_{lm}$ **num** + *T*	F → num
$\Rightarrow_{lm}$ **num** + *T* * *F*	T → T * F
$\Rightarrow_{lm}$ **num** + *F* * *F*	T → F
$\Rightarrow_{lm}$ **num** + **num** * *F*	F → num
$\Rightarrow_{lm}$ **num** + **num** * **num**	F → num

5.5 Implementación y pruebas

El parser se genera con ANTLR 4.13.1 para Python 3 con la opción `-visitor` (`make generate`). Cambiamos los *listeners* de error que trae ANTLR por uno propio que guarda cada mensaje con su línea y columna. Si hay al menos uno, el driver los imprime y termina con código 1, sin pasar al análisis semántico. El Listado 6 corresponde a un programa al que le falta un punto y coma y el nombre de un proceso. Los mensajes de ANTLR siguen en inglés; traducirlos queda pendiente.

Listado 6
Errores sintácticos en tests/error_sintactico.txt

```
=== ERRORES LÉXICOS / SINTÁCTICOS ===
  [Error Sintáctico, línea 2:0] missing ';' at 'proceso'
  [Error Sintáctico, línea 2:8] missing ID at '{'
```

6. Analizador semántico

6.1 Tabla de símbolos

El análisis semántico es un visitor que recorre el árbol una sola vez y en orden. Por eso un proceso tiene que aparecer antes de la sentencia *generar* que lo usa. Cada símbolo guarda nombre, tipo (recurso, proceso, etapa o métrica), ámbito, línea, valor y si llegó a usarse. Hay un ámbito global y uno por proceso, y la búsqueda va del ámbito actual hacia el global.

En la primera versión los ámbitos se identificaban por el nombre del proceso. Un proceso llamado *global* terminaba escribiendo sus etapas en el ámbito global, y `etapa m usa m` daba un error de duplicado que no correspondía. Ahora cada ámbito tiene un número interno y el nombre solo aparece en los mensajes.

Las expresiones se evalúan durante el análisis. Un recurso que aparece dentro de una expresión aporta su capacidad, de modo que `recurso n = m - 2;` se rechaza cuando *m* vale 1.

6.2 Errores que detecta

El Hito 2 pide seis. El compilador comprueba 22, listados en la Tabla 10. Los mensajes no están escritos a mano: un script ejecuta el compilador sobre un programa mínimo que solo contiene ese error y copia lo que imprime, así el informe no se desactualiza cuando cambia el código.

Tabla 10
Verificaciones semánticas y mensaje que produce cada una

N.°	Verificación	Ejemplo	Mensaje del compilador
1	Identificador duplicado en el mismo ámbito	`recurso m = 2;`	[línea 2] El identificador 'm' ya ha sido declarado en el ámbito 'global'.
2	Recurso no declarado en una etapa	`etapa e usa enfermeros durante 5 minutos;`	[línea 2] Recurso 'enfermeros' utilizado en etapa 'e' no está declarado
3	Una etapa usa algo que no es recurso	`etapa x usa p durante 5 minutos;`	[línea 3] 'p' es de tipo 'proceso'; una etapa solo puede usar un recurso
4	Proceso destino no declarado	`... hacia fantasma;`	[línea 4] El proceso de destino 'fantasma' no ha sido declarado
5	El destino de una llegada no es un proceso	`... hacia m;`	[línea 4] 'm' no es un proceso válido de destino
6	Identificador no declarado en una expresión	`recurso n = x + 1;`	[línea 2] El identificador 'x' no está declarado

N.°	Verificación	Ejemplo	Mensaje del compilador
7	Identificador que no es recurso en una expresión	`recurso n = p * 2;`	[línea 3] 'p' es de tipo 'proceso'; en una expresión solo se pueden usar recursos
8	División entre cero	`recurso n = 4 / (2 - 2);`	[línea 2] División entre cero
9	Número no finito	`recurso n = 1000…0; (401 cifras)`	[línea 2] El valor numérico debe ser finito
10	Capacidad menor o igual a 0	`recurso n = 2 - 5;`	[línea 2] La capacidad del recurso 'n' debe ser mayor a 0
11	Capacidad no entera	`recurso n = 3 / 2;`	[línea 2] La capacidad del recurso 'n' debe ser un número entero (se obtuvo 1.5)
12	Duración de etapa menor o igual a 0	`etapa e usa m durante 0 minutos;`	[línea 2] La duración de la etapa 'e' debe ser mayor a 0
13	Intervalo de llegada menor o igual a 0	`generar pacientes cada 0 minutos hacia p;`	[línea 3] El intervalo de llegada de pacientes debe ser mayor a 0
14	Duración de simulación menor o igual a 0	`simular durante 0 horas;`	[línea 4] La duración de la simulación debe ser mayor a 0
15	Proceso sin etapas	`proceso q { }`	[línea 3] El proceso 'q' no tiene etapas
16	Falta la sentencia simular	`(programa sin simular)`	[línea 3] Falta la sentencia 'simular durante ...'
17	Más de una sentencia simular	`simular … (por segunda vez)`	[línea 5] Solo se permite una sentencia 'simular'
18	mostrar sobre algo que no es métrica	`mostrar m;`	[línea 5] 'm' no es una métrica (use pacientes_atendidos, espera_promedio u ocupacion)
19	ocupacion sin recurso	`mostrar ocupacion;`	[línea 5] La métrica 'ocupacion' requiere un recurso: ocupacion(recurso)
20	Métrica que no recibe argumentos	`mostrar espera_promedio(m);`	[línea 5] La métrica 'espera_promedio' no recibe argumentos
21	ocupacion sobre algo que no es recurso	`mostrar ocupacion(p);`	[línea 5] No se puede medir ocupación sobre 'p' porque no es un recurso
22	ocupacion sobre un recurso inexistente	`mostrar ocupacion(rayos_x);`	[línea 5] El recurso 'rayos_x' no existe para la consulta de ocupación

Nota. Cada ejemplo se insertó en un programa válido de cuatro líneas; la línea indicada corresponde a ese programa.

Una de estas reglas necesitó dos intentos. La comprobación de capacidad entera aceptaba cualquier valor a menos de 10^{-9} de un entero, para no rechazar `0.1 * 30`, que en coma flotante da 3.0000000000000004. Pero 0.0000000001 también está a menos de 10^{-9} de un entero, el cero: pasaba la validación, se redondeaba a 0 y el simulador fallaba con un recurso sin servidores. Ahora se exige además que el valor redondeado sea al menos 1, y el mismo programa produce este mensaje:

Listado 7

Capacidad casi cero

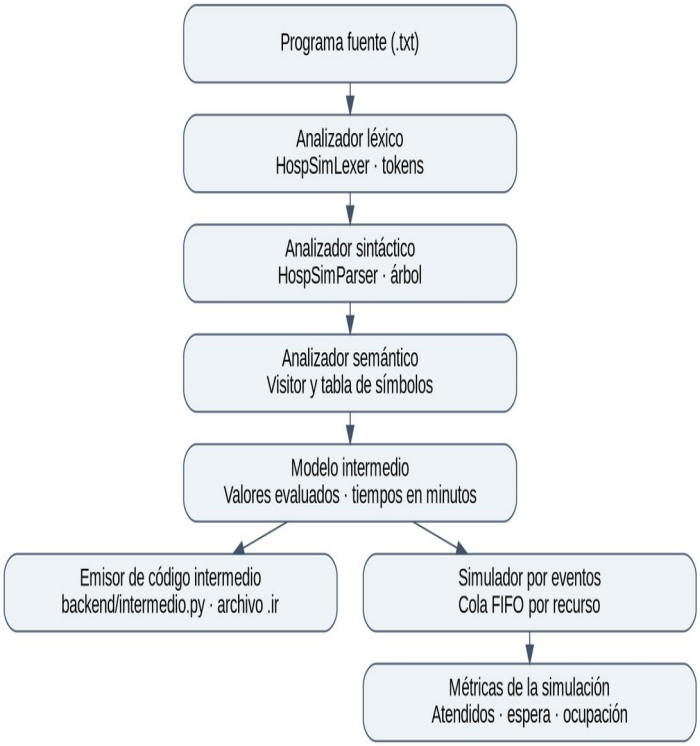
```
[Error Semántico, línea 1] La capacidad del recurso 'm' debe ser un número entero
(se obtuvo 1e-10)
```

Los recursos y procesos declarados que nunca se usan generan una advertencia. La advertencia no detiene la compilación; un error sí, con código de salida 1 y sin ejecutar el back end.

7. Arquitectura del compilador

La Figura 1 muestra las fases y la Tabla 11, el módulo de código que implementa cada una.

Figura 1
Fases del compilador HospSim



Nota. Si una fase encuentra errores, el driver los imprime y no ejecuta las siguientes.

Tabla 11
Módulos del repositorio

Módulo	Qué hace
`HospSim.g4`	Gramática léxica y sintáctica.
`gen/`	Lexer, parser y visitor generados por ANTLR 4.13.1.
`main.py`	Encadena las fases, reporta errores con línea y columna y define el código de salida.
`semantic/symbol_table.py`	Tabla de símbolos con pila de ámbitos.
`semantic/ semantic_visitor.py`	Verificaciones semánticas y evaluación de expresiones.
`backend/modelo.py`	Convierte el árbol validado en el Modelo.
`backend/intermedio.py`	Escribe el Modelo como código intermedio (`--ir`).
`backend/simulador.py`	Ejecuta el Modelo por eventos discretos y calcula las métricas.
`tools/derivaciones.py`	Genera las derivaciones del Anexo A desde el parser.
`tests/, run_tests.py`	Casos con salida esperada y pruebas de regresión.

7.1 Decisiones de diseño

Separar el análisis semántico de la construcción del modelo cuesta un segundo recorrido del árbol, pero deja un back end más simple: `ConstructorModelo` solo recibe árboles válidos y no repite comprobaciones.

El Modelo es una estructura de Python sin referencias a ANTLR. Contiene los recursos con su capacidad, los procesos con sus etapas, las llegadas, la duración y las consultas, con todos los tiempos en minutos. De él salen dos cosas: `backend/intermedio.py` lo escribe como código intermedio y `backend/simulador.py` lo ejecuta. El simulador no lee el archivo .ir; los dos parten del mismo Modelo en memoria. Según el alcance aprobado para el proyecto, el compilador genera este código intermedio propio y no LLVM.

El simulador cambió durante este avance. La primera versión atendía a cada paciente de principio a fin antes de pasar al siguiente. Era fácil de comprobar a mano, pero con recursos compartidos daba resultados falsos: un paciente que había llegado antes reservaba un médico para más tarde y bloqueaba a otro que podía usarlo en ese momento. En un caso con dos procesos reportaba 20 minutos de espera promedio donde nadie espera. La versión actual procesa los eventos en orden de tiempo con un montículo binario (Python Software Foundation, s. f.) y mantiene una cola FIFO por recurso. Un paciente pide un recurso recién cuando llega a la etapa que lo usa.

El resultado depende de unas pocas reglas, que dejamos fijas para que las salidas sean reproducibles. Las llegadas ocurren en t = 0, k, 2k, … mientras t sea menor que la duración. A igual tiempo se procesan primero las finalizaciones y después las llegadas; entre eventos de la misma clase manda el orden en que se crearon. Un paciente que termina una etapa y vuelve a pedir el mismo recurso se pone detrás de quienes ya esperaban. Cuenta como atendido si termina dentro del horizonte, incluido el instante final. La espera promedio se calcula solo sobre los atendidos, y la ocupación suma también el servicio parcial que cae dentro del horizonte.

7.2 Código intermedio

`python main.py --ir ejemplos/atencion.txt` imprime el código intermedio del Listado 1, que aparece en el Listado 8; `--guardar-ir archivo` lo escribe en disco. La primera parte declara recursos, colas, procesos, llegadas, duración y reportes, y termina en HALT. Luego viene una etiqueta por proceso con sus etapas: ACQUIRE pide una unidad del recurso, DELAY la retiene el tiempo de la etapa y RELEASE la devuelve.

Listado 8
Código intermedio del consultorio con un médico

```
; HospSim IR 1 - tiempos en minutos
RESOURCE medicos 1
QUEUE medicos FIFO
PROCESS atencion atencion_inicio
ARRIVALS atencion EVERY 5 FROM 0
RUN_UNTIL 60
REPORT pacientes_atendidos
REPORT espera_promedio
REPORT ocupacion medicos
HALT
atencion_inicio:
  ; etapa consulta
  ACQUIRE medicos
  DELAY 15
  RELEASE medicos
  COMPLETE
```

8. Plan de validación

8.1 Método

Validamos con pruebas de caja negra. Cada caso es un programa en `tests/` con su salida esperada en `tests/esperado/`. El comando `python run_tests.py` compila los 20 casos, compara salida y código de retorno y después corre las 11 pruebas de regresión de `tests/test_regresiones.py`. Funciona igual en Windows que en Linux.

Seguimos una regla: ninguna salida esperada se acepta hasta que alguien del grupo la calcula a mano. De otro modo la prueba solo confirmaría lo que el compilador ya hace, errores incluidos.

8.2 Casos

Tabla 12
Casos de prueba por fase

Fase	Casos	Qué comprueba
Léxica	error_lexico	Carácter inválido con línea y columna.
Sintáctica	error_sintactico	Punto y coma y nombre de proceso faltantes.
Semántica	entrada_error, error_capacidad, error_capacidad_entera, error_duplicado, error_estructura, error_expr, error_metricas, error_simular_doble, error_tiempos, error_tipo_expresion, error_tipos, error_vacio	Mensajes de error y advertencias de símbolos sin usar.
Gramática	sim_precedencia, sim_asociatividad	2 + 3 * 4 = 14 y 20 − 5 − 3 = 12.
Back end	entrada_correcta, sim_dos_servidores, sim_dos_etapas, sim_precedencia, sim_asociatividad, entrada_completa	Métricas contra cálculos a mano.

Fase	Casos	Qué comprueba
Regresión	capacidad 0.0000000001 rechazada; un programa con errores no emite código intermedio; cola FIFO en un recurso compartido; límites del horizonte; código intermedio exacto y guardado en archivo; número no finito rechazado; proceso llamado global; recurso compartido sin reservas a futuro; 0.1 * 30 aceptado como 3; volver a pedir el mismo recurso respeta la cola; programa sin llegadas	Fallas encontradas en la revisión del simulador y del semántico.

8.3 Resultados calculados a mano

La Tabla 13 resume la cuenta de cada caso del back end. Estos seis resultados no cambiaron al reemplazar el simulador: en ninguno de ellos un paciente podría adelantar a otro, y en ese escenario ambas versiones coinciden.

Tabla 13
Cálculo manual de las métricas

Caso	Cálculo	Resultado
`entrada_correcta`	1 médico, consulta de 15 min, llegada cada 5 min, 60 min. Atenciones 0–15, 15–30, 30–45 y 45–60; la quinta termina en 75. Esperas 0, 10, 20 y 30.	4 atendidos; espera 15.00; ocupación 100.00 %
`sim_dos_servidores`	Igual, con 2 médicos. Servidor A ocupado de 0 a 60; B de 5 a 65, de los que 55 min caen dentro del horizonte. Esperas 0, 0, 5, 5, 10, 10 y 15.	7; 45/7 = 6.43; 115/120 = 95.83 %
`sim_dos_etapas`	Triaje de 5 min (1 enfermero) y consulta de 12 min (1 médico), llegada cada 10 min. Consultas 5–17, 17–29, 29–41 y 41–53; esperas 0, 2, 4 y 6.	4; 3.00; 50.00 % y 55/60 = 91.67 %
`sim_precedencia`	14 camas, 60 min por paciente, llegada cada 30 min, 120 min. Terminan en 60, 90, 120 y 150. Con la gramática del Hito 1 serían 20 camas y 8.75 %.	3; 210/1680 = 12.50 %
`sim_asociatividad`	12 médicos y 3 camas, llegada cada hora, 180 min. Los médicos suman 180 min de uso y las camas 60.	2; 180/2160 = 8.33 %; 60/540 = 11.11 %
`entrada_completa`	2 médicos (10 min) y 5 camas (60 min), llegada cada 5 min, 480 min. El paciente i = 5q + r entra a cama en 5r + 10 + 60q y espera 35q; terminan a tiempo los de q = 0 a 6.	35; 105.00; 955/960 = 99.48 %; 2300/2400 = 95.83 %

Al simulador nuevo le hicimos una comprobación adicional. Generamos 3000 modelos aleatorios de un proceso con una sola etapa, donde el algoritmo anterior es exacto, y los dos simuladores dieron los mismos resultados en todos. En otros 3000 modelos con recursos compartidos verificamos que la ocupación quedara entre 0 y 100 % y que nunca hubiera más atendidos que llegadas. También calculamos a mano un caso con dos procesos que comparten un médico, uno de los cuales lo usa dos veces seguidas: 4 atendidos, espera de 1.50 minutos, 90 % de ocupación del médico y 25 % del enfermero. El simulador dio lo mismo.

8.4 Resultados de la ejecución

Con la versión entregada pasan los 20 casos y las 11 regresiones (Listado 9).

Listado 9
Salida de python run_tests.py

```
OK     entrada_completa
OK     entrada_correcta
OK     entrada_error
OK     error_capacidad
OK     error_capacidad_entera
OK     error_duplicado
OK     error_estructura
OK     error_expr
OK     error_lexico
OK     error_metricas
OK     error_simular_doble
OK     error_sintactico
OK     error_tiempos
OK     error_tipo_expresion
OK     error_tipos
OK     error_vacio
OK     sim_asociatividad
OK     sim_dos_etapas
OK     sim_dos_servidores
OK     sim_precedencia
20 casos de salida comprobados; 0 bloque(s) fallido(s).
Ran 11 tests ... OK
```

Aceptamos una versión solo si pasan los 20 casos y las 11 regresiones. Cada verificación semántica nueva o cambio de gramática entra junto con su caso de prueba.

9. Estado del avance

Tabla 14
Estado de cada componente

Componente	Estado
Analizador léxico (ANTLR4)	Terminado
Analizador sintáctico con gramática estratificada	Terminado
Analizador semántico (22 verificaciones, ámbitos por proceso)	Terminado
Modelo intermedio y código intermedio textual (--ir)	Terminado
Simulador por eventos con cola FIFO por recurso	Terminado
Pruebas automáticas	20 casos y 11 regresiones, todos en OK

Conviene dejar escrito también lo que HospSim no hace. La simulación es determinista: no hay tiempos aleatorios, prioridades clínicas ni recorridos que dependan del paciente. Un proceso solo puede usar recursos declarados antes que él. Los mensajes sintácticos siguen en inglés. Para la entrega final vamos a ampliar los escenarios de validación y medir cuánto tarda la simulación con modelos grandes.

Referencias

Aho, A. V., Lam, M. S., Sethi, R. y Ullman, J. D. (2007). *Compilers: Principles, techniques, and tools* (2.ª ed.). Pearson.

Banks, J., Carson, J. S., Nelson, B. L. y Nicol, D. M. (2010). *Discrete-event system simulation* (5.ª ed.). Pearson.

Parr, T. (2013). *The definitive ANTLR 4 reference*. The Pragmatic Bookshelf.

Python Software Foundation. (s. f.). *heapq: Heap queue algorithm*. Python 3 documentation. https://docs.python.org/3/library/heapq.html

Soncco Álvarez, J. L. (s. f.). *Construcción de compiladores. Unidad II: Análisis sintáctico* [Diapositivas de clase]. Universidad Peruana de Ciencias Aplicadas.

Team SimPy. (s. f.). *SimPy: Discrete event simulation for Python*. https://simpy.readthedocs.io

Anexo A. Derivaciones más a la izquierda

Las genera `tools/derivaciones.py` a partir del árbol que construye el parser de ANTLR, convertido a la gramática G de la Tabla 9. El script se detiene si alguna derivación usa una producción que no está en G. A la derecha de cada paso va la producción aplicada.

A.1 Programa

Entrada: `simular durante 8 horas;`

programa	
$\Rightarrow_{lm}$ *sentencias*	programa → sentencias
$\Rightarrow_{lm}$ *sentencia sentencias*	sentencias → sentencia sentencias
$\Rightarrow_{lm}$ *sent_simular sentencias*	sentencia → sent_simular
$\Rightarrow_{lm}$ **simular durante** *E unidad* ; *sentencias*	sent_simular → simular durante E unidad ;
$\Rightarrow_{lm}$ **simular durante** *T unidad* ; *sentencias*	E → T
$\Rightarrow_{lm}$ **simular durante** *F unidad* ; *sentencias*	T → F
$\Rightarrow_{lm}$ **simular durante num** *unidad* ; *sentencias*	F → num
$\Rightarrow_{lm}$ **simular durante num horas** ; *sentencias*	unidad → horas
$\Rightarrow_{lm}$ **simular durante num horas** ;	sentencias → ε

Entrada: `recurso medicos = 2;  simular durante 1 hora;`

programa	
$\Rightarrow_{lm}$ *sentencias*	programa → sentencias
$\Rightarrow_{lm}$ *sentencia sentencias*	sentencias → sentencia sentencias
$\Rightarrow_{lm}$ *decl_recurso sentencias*	sentencia → decl_recurso
$\Rightarrow_{lm}$ **recurso id** = *E* ; *sentencias*	decl_recurso → recurso id = E ;
$\Rightarrow_{lm}$ **recurso id** = *T* ; *sentencias*	E → T
$\Rightarrow_{lm}$ **recurso id** = *F* ; *sentencias*	T → F
$\Rightarrow_{lm}$ **recurso id** = **num** ; *sentencias*	F → num
$\Rightarrow_{lm}$ **recurso id** = **num** ; *sentencia sentencias*	sentencias → sentencia sentencias
$\Rightarrow_{lm}$ **recurso id** = **num** ; *sent_simular sentencias*	sentencia → sent_simular
$\Rightarrow_{lm}$ **recurso id** = **num** ; **simular durante** *E unidad* ; *sentencias*	sent_simular → simular durante E unidad ;
$\Rightarrow_{lm}$ **recurso id** = **num** ; **simular durante** *T unidad* ; *sentencias*	E → T
$\Rightarrow_{lm}$ **recurso id** = **num** ; **simular durante** *F unidad* ; *sentencias*	T → F
$\Rightarrow_{lm}$ **recurso id** = **num** ; **simular durante num** *unidad* ; *sentencias*	F → num
$\Rightarrow_{lm}$ **recurso id** = **num** ; **simular durante num hora** ; *sentencias*	unidad → hora
$\Rightarrow_{lm}$ **recurso id** = **num** ; **simular durante num hora** ;	sentencias → ε

Entrada: `recurso camas = 10;  mostrar ocupacion(camas);`

programa

$\Rightarrow_{lm}$ *sentencias*	programa → sentencias
$\Rightarrow_{lm}$ *sentencia sentencias*	sentencias → sentencia sentencias
$\Rightarrow_{lm}$ *decl_recurso sentencias*	sentencia → decl_recurso
$\Rightarrow_{lm}$ **recurso id** = *E* ; *sentencias*	decl_recurso → recurso id = E ;
$\Rightarrow_{lm}$ **recurso id** = *T* ; *sentencias*	E → T
$\Rightarrow_{lm}$ **recurso id** = *F* ; *sentencias*	T → F
$\Rightarrow_{lm}$ **recurso id** = **num** ; *sentencias*	F → num
$\Rightarrow_{lm}$ **recurso id** = **num** ; *sentencia sentencias*	sentencias → sentencia sentencias
$\Rightarrow_{lm}$ **recurso id** = **num** ; *sent_mostrar sentencias*	sentencia → sent_mostrar
$\Rightarrow_{lm}$ **recurso id** = **num** ; **mostrar id (id)** ; *sentencias*	sent_mostrar → mostrar id (id) ;
$\Rightarrow_{lm}$ **recurso id** = **num** ; **mostrar id (id)** ;	sentencias → ε

Entrada: `proceso p { }  generar pacientes cada 5 minutos hacia p;`

programa

$\Rightarrow_{lm}$ *sentencias*	programa → sentencias
$\Rightarrow_{lm}$ *sentencia sentencias*	sentencias → sentencia sentencias
$\Rightarrow_{lm}$ *decl_proceso sentencias*	sentencia → decl_proceso
$\Rightarrow_{lm}$ **proceso id** { *etapas* } *sentencias*	decl_proceso → proceso id { etapas }
$\Rightarrow_{lm}$ **proceso id** { } *sentencias*	etapas → ε
$\Rightarrow_{lm}$ **proceso id** { } *sentencia sentencias*	sentencias → sentencia sentencias
$\Rightarrow_{lm}$ **proceso id** { } *sent_llegada sentencias*	sentencia → sent_llegada
$\Rightarrow_{lm}$ **proceso id** { } **generar pacientes cada** *E unidad* **hacia id** ; *sentencias*	sent_llegada → generar pacientes cada E unidad hacia id ;
$\Rightarrow_{lm}$ **proceso id** { } **generar pacientes cada** *T unidad* **hacia id** ; *sentencias*	E → T
$\Rightarrow_{lm}$ **proceso id** { } **generar pacientes cada** *F unidad* **hacia id** ; *sentencias*	T → F
$\Rightarrow_{lm}$ **proceso id** { } **generar pacientes cada num** *unidad* **hacia id** ; *sentencias*	F → num
$\Rightarrow_{lm}$ **proceso id** { } **generar pacientes cada num minutos hacia id** ; *sentencias*	unidad → minutos
$\Rightarrow_{lm}$ **proceso id** { } **generar pacientes cada num minutos hacia id** ;	sentencias → ε

A.2 Declaración de recurso

Entrada: `recurso medicos = 3;`

decl_recurso

$\Rightarrow_{lm}$ **recurso id** = *E* ;	decl_recurso → recurso id = E ;
$\Rightarrow_{lm}$ **recurso id** = *T* ;	E → T
$\Rightarrow_{lm}$ **recurso id** = *F* ;	T → F
$\Rightarrow_{lm}$ **recurso id** = **num** ;	F → num

Entrada: `recurso camas = 2 * 10;`

decl_recurso

$\Rightarrow_{lm}$ **recurso id** = *E* ;	decl_recurso → recurso id = E ;
$\Rightarrow_{lm}$ **recurso id** = *T* ;	E → T
$\Rightarrow_{lm}$ **recurso id** = *T* * *F* ;	T → T * F
$\Rightarrow_{lm}$ **recurso id** = *F* * *F* ;	T → F
$\Rightarrow_{lm}$ **recurso id** = **num** * *F* ;	F → num
$\Rightarrow_{lm}$ **recurso id** = **num** * **num** ;	F → num

Entrada: `recurso sillas = medicos + 4;`

decl_recurso

$\Rightarrow_{lm}$ **recurso id** = *E* ;	decl_recurso → recurso id = E ;
$\Rightarrow_{lm}$ **recurso id** = *E* + *T* ;	E → E + T
$\Rightarrow_{lm}$ **recurso id** = *T* + *T* ;	E → T
$\Rightarrow_{lm}$ **recurso id** = *F* + *T* ;	T → F
$\Rightarrow_{lm}$ **recurso id** = **id** + *T* ;	F → id
$\Rightarrow_{lm}$ **recurso id** = **id** + *F* ;	T → F
$\Rightarrow_{lm}$ **recurso id** = **id** + **num** ;	F → num

Entrada: `recurso boxes = (2 + 3) * 2;`

decl_recurso

$\Rightarrow_{lm}$ **recurso id** = *E* ;	decl_recurso → recurso id = E ;
$\Rightarrow_{lm}$ **recurso id** = *T* ;	E → T
$\Rightarrow_{lm}$ **recurso id** = *T* * *F* ;	T → T * F
$\Rightarrow_{lm}$ **recurso id** = *F* * *F* ;	T → F
$\Rightarrow_{lm}$ **recurso id** = (*E*) * *F* ;	F → (E)
$\Rightarrow_{lm}$ **recurso id** = (*E* + *T*) * *F* ;	E → E + T
$\Rightarrow_{lm}$ **recurso id** = (*T* + *T*) * *F* ;	E → T
$\Rightarrow_{lm}$ **recurso id** = (*F* + *T*) * *F* ;	T → F
$\Rightarrow_{lm}$ **recurso id** = (**num** + *T*) * *F* ;	F → num
$\Rightarrow_{lm}$ **recurso id** = (**num** + *F*) * *F* ;	T → F
$\Rightarrow_{lm}$ **recurso id** = (**num** + **num**) * *F* ;	F → num
$\Rightarrow_{lm}$ **recurso id** = (**num** + **num**) * **num** ;	F → num

A.3 Declaración de proceso con sus etapas

Entrada: `proceso vacio { }`

decl_proceso

$\Rightarrow_{lm}$ **proceso id** { *etapas* }	decl_proceso → proceso id { etapas }
$\Rightarrow_{lm}$ **proceso id** { }	etapas → ε

Entrada: `proceso atencion { etapa consulta usa medicos durante 15 minutos; }`

decl_proceso

⇒lm **proceso id** { *etapas* }	decl_proceso → proceso id { etapas }
⇒lm **proceso id** { *decl_etapa etapas* }	etapas → decl_etapa etapas
⇒lm **proceso id** { **etapa id usa id durante** *E unidad* ; *etapas* }	decl_etapa → etapa id usa id durante E unidad ;
⇒lm **proceso id** { **etapa id usa id durante** *T unidad* ; *etapas* }	E → T
⇒lm **proceso id** { **etapa id usa id durante** *F unidad* ; *etapas* }	T → F
⇒lm **proceso id** { **etapa id usa id durante num** *unidad* ; *etapas* }	F → num
⇒lm **proceso id** { **etapa id usa id durante num minutos** ; *etapas* }	unidad → minutos
⇒lm **proceso id** { **etapa id usa id durante num minutos** ; }	etapas → ε

Entrada: `proceso emergencia { etapa triaje usa enfermeros durante 5 minutos; etapa reanimacion usa medicos durante 1 hora; }`

decl_proceso

⇒lm **proceso id** { *etapas* }	decl_proceso → proceso id { etapas }
⇒lm **proceso id** { *decl_etapa etapas* }	etapas → decl_etapa etapas
⇒lm **proceso id** { **etapa id usa id durante** *E unidad* ; *etapas* }	decl_etapa → etapa id usa id durante E unidad ;
⇒lm **proceso id** { **etapa id usa id durante** *T unidad* ; *etapas* }	E → T
⇒lm **proceso id** { **etapa id usa id durante** *F unidad* ; *etapas* }	T → F
⇒lm **proceso id** { **etapa id usa id durante num** *unidad* ; *etapas* }	F → num
⇒lm **proceso id** { **etapa id usa id durante num minutos** ; *etapas* }	unidad → minutos
⇒lm **proceso id** { **etapa id usa id durante num minutos** ; *decl_etapa etapas* }	etapas → decl_etapa etapas
⇒lm **proceso id** { **etapa id usa id durante num minutos** ; **etapa id usa id durante** *E unidad* ; *etapas* }	decl_etapa → etapa id usa id durante E unidad ;
⇒lm **proceso id** { **etapa id usa id durante num minutos** ; **etapa id usa id durante** *T unidad* ; *etapas* }	E → T
⇒lm **proceso id** { **etapa id usa id durante num minutos** ; **etapa id usa id durante** *F unidad* ; *etapas* }	T → F
⇒lm **proceso id** { **etapa id usa id durante num minutos** ; **etapa id usa id durante num** *unidad* ; *etapas* }	F → num
⇒lm **proceso id** { **etapa id usa id durante num minutos** ; **etapa id usa id durante num hora** ; *etapas* }	unidad → hora
⇒lm **proceso id** { **etapa id usa id durante num minutos** ; **etapa id usa id durante num hora** ; }	etapas → ε

Entrada: `proceso alta { etapa tramite usa admision durante 10 / 2 minutos; }`

decl_proceso

⇒lm **proceso id** { *etapas* }	decl_proceso → proceso id { etapas }
⇒lm **proceso id** { *decl_etapa etapas* }	etapas → decl_etapa etapas
⇒lm **proceso id** { **etapa id usa id durante** *E unidad* ; *etapas* }	decl_etapa → etapa id usa id durante E unidad ;
⇒lm **proceso id** { **etapa id usa id durante** *T unidad* ; *etapas* }	E → T
⇒lm **proceso id** { **etapa id usa id durante** *T* / *F unidad* ; *etapas* }	T → T / F
⇒lm **proceso id** { **etapa id usa id durante** *F* / *F unidad* ; *etapas* }	T → F
⇒lm **proceso id** { **etapa id usa id durante num** / *F unidad* ; *etapas* }	F → num
⇒lm **proceso id** { **etapa id usa id durante num** / **num** *unidad* ; *etapas* }	F → num
⇒lm **proceso id** { **etapa id usa id durante num** / **num minutos** ; *etapas* }	unidad → minutos
⇒lm **proceso id** { **etapa id usa id durante num** / **num minutos** ; }	etapas → ε

A.4 Generación de llegadas

Entrada: `generar pacientes cada 5 minutos hacia atencion;`

sent_llegada

⇒lm **generar pacientes cada** *E unidad* **hacia id** ;	sent_llegada → generar pacientes cada E unidad hacia id ;
⇒lm **generar pacientes cada** *T unidad* **hacia id** ;	E → T
⇒lm **generar pacientes cada** *F unidad* **hacia id** ;	T → F
⇒lm **generar pacientes cada num** *unidad* **hacia id** ;	F → num
⇒lm **generar pacientes cada num minutos hacia id** ;	unidad → minutos

Entrada: `generar pacientes cada 1 hora hacia emergencia;`

sent_llegada

⇒lm **generar pacientes cada** *E unidad* **hacia id** ;	sent_llegada → generar pacientes cada E unidad hacia id ;
⇒lm **generar pacientes cada** *T unidad* **hacia id** ;	E → T
⇒lm **generar pacientes cada** *F unidad* **hacia id** ;	T → F
⇒lm **generar pacientes cada num** *unidad* **hacia id** ;	F → num
⇒lm **generar pacientes cada num hora hacia id** ;	unidad → hora

Entrada: `generar pacientes cada 2 * 3 minutos hacia triaje;`

sent_llegada

⇒lm **generar pacientes cada** *E unidad* **hacia id** ;	sent_llegada → generar pacientes cada E unidad hacia id ;
⇒lm **generar pacientes cada** *T unidad* **hacia id** ;	E → T
⇒lm **generar pacientes cada** *T* * *F unidad* **hacia id** ;	T → T * F
⇒lm **generar pacientes cada** *F* * *F unidad* **hacia id** ;	T → F
⇒lm **generar pacientes cada num** * *F unidad* **hacia id** ;	F → num
⇒lm **generar pacientes cada num * num** *unidad* **hacia id** ;	F → num
⇒lm **generar pacientes cada num * num minutos hacia id** ;	unidad → minutos

Entrada: `generar pacientes cada 0.5 horas hacia consulta_externa;`

sent_llegada

⇒lm **generar pacientes cada** *E unidad* **hacia id** ;	sent_llegada → generar pacientes cada E unidad hacia id ;
⇒lm **generar pacientes cada** *T unidad* **hacia id** ;	E → T
⇒lm **generar pacientes cada** *F unidad* **hacia id** ;	T → F
⇒lm **generar pacientes cada num** *unidad* **hacia id** ;	F → num
⇒lm **generar pacientes cada num horas hacia id** ;	unidad → horas

A.5 Simulación

Entrada: `simular durante 8 horas;`

sent_simular

⇒lm **simular durante** *E unidad* ;	sent_simular → simular durante E unidad ;
⇒lm **simular durante** *T unidad* ;	E → T
⇒lm **simular durante** *F unidad* ;	T → F
⇒lm **simular durante num** *unidad* ;	F → num
⇒lm **simular durante num horas** ;	unidad → horas

Entrada: `simular durante 90 minutos;`

sent_simular

⇒lm **simular durante** *E unidad* ;	sent_simular → simular durante E unidad ;
⇒lm **simular durante** *T unidad* ;	E → T
⇒lm **simular durante** *F unidad* ;	T → F
⇒lm **simular durante num** *unidad* ;	F → num
⇒lm **simular durante num minutos** ;	unidad → minutos

Entrada: `simular durante 1 hora;`

sent_simular

⇒lm **simular durante** *E unidad* ;	sent_simular → simular durante E unidad ;
⇒lm **simular durante** *T unidad* ;	E → T
⇒lm **simular durante** *F unidad* ;	T → F
⇒lm **simular durante num** *unidad* ;	F → num
⇒lm **simular durante num hora** ;	unidad → hora

Entrada: `simular durante 24 * 7 horas;`

sent_simular

$\Rightarrow_{lm}$ **simular durante** *E unidad* ;	sent_simular → simular durante E unidad ;
$\Rightarrow_{lm}$ **simular durante** *T unidad* ;	E → T
$\Rightarrow_{lm}$ **simular durante** *T* * *F unidad* ;	T → T * F
$\Rightarrow_{lm}$ **simular durante** *F* * *F unidad* ;	T → F
$\Rightarrow_{lm}$ **simular durante num** * *F unidad* ;	F → num
$\Rightarrow_{lm}$ **simular durante num * num** *unidad* ;	F → num
$\Rightarrow_{lm}$ **simular durante num * num horas** ;	unidad → horas

A.6 Consulta de métricas

Entrada: `mostrar pacientes_atendidos;`

sent_mostrar

$\Rightarrow_{lm}$ **mostrar id** ;	sent_mostrar → mostrar id ;

Entrada: `mostrar espera_promedio;`

sent_mostrar

$\Rightarrow_{lm}$ **mostrar id** ;	sent_mostrar → mostrar id ;

Entrada: `mostrar ocupacion(medicos);`

sent_mostrar

$\Rightarrow_{lm}$ **mostrar id (id)** ;	sent_mostrar → mostrar id (id) ;

Entrada: `mostrar ocupacion(camas);`

sent_mostrar

$\Rightarrow_{lm}$ **mostrar id (id)** ;	sent_mostrar → mostrar id (id) ;

A.7 Expresiones aritméticas

Entrada: `2 + 3 * 4`

E

$\Rightarrow_{lm}$ *E* + *T*	E → E + T
$\Rightarrow_{lm}$ *T* + *T*	E → T
$\Rightarrow_{lm}$ *F* + *T*	T → F
$\Rightarrow_{lm}$ **num** + *T*	F → num
$\Rightarrow_{lm}$ **num** + *T* * *F*	T → T * F
$\Rightarrow_{lm}$ **num** + *F* * *F*	T → F
$\Rightarrow_{lm}$ **num** + **num** * *F*	F → num
$\Rightarrow_{lm}$ **num** + **num** * **num**	F → num

Entrada: `(2 + 3) * 4`

E

$\Rightarrow_{lm} T$ — E → T

$\Rightarrow_{lm} T * F$ — T → T * F

$\Rightarrow_{lm} F * F$ — T → F

$\Rightarrow_{lm} (E) * F$ — F → (E)

$\Rightarrow_{lm} (E + T) * F$ — E → E + T

$\Rightarrow_{lm} (T + T) * F$ — E → T

$\Rightarrow_{lm} (F + T) * F$ — T → F

$\Rightarrow_{lm} (\mathbf{num} + T) * F$ — F → num

$\Rightarrow_{lm} (\mathbf{num} + F) * F$ — T → F

$\Rightarrow_{lm} (\mathbf{num} + \mathbf{num}) * F$ — F → num

$\Rightarrow_{lm} (\mathbf{num} + \mathbf{num}) * \mathbf{num}$ — F → num

Entrada: `20 - 5 - 3`

E

$\Rightarrow_{lm} E - T$ — E → E − T

$\Rightarrow_{lm} E - T - T$ — E → E − T

$\Rightarrow_{lm} T - T - T$ — E → T

$\Rightarrow_{lm} F - T - T$ — T → F

$\Rightarrow_{lm} \mathbf{num} - T - T$ — F → num

$\Rightarrow_{lm} \mathbf{num} - F - T$ — T → F

$\Rightarrow_{lm} \mathbf{num} - \mathbf{num} - T$ — F → num

$\Rightarrow_{lm} \mathbf{num} - \mathbf{num} - F$ — T → F

$\Rightarrow_{lm} \mathbf{num} - \mathbf{num} - \mathbf{num}$ — F → num

Entrada: `medicos / 2`

E

$\Rightarrow_{lm} T$ — E → T

$\Rightarrow_{lm} T / F$ — T → T / F

$\Rightarrow_{lm} F / F$ — T → F

$\Rightarrow_{lm} \mathbf{id} / F$ — F → id

$\Rightarrow_{lm} \mathbf{id} / \mathbf{num}$ — F → num

Anexo B. Bitácora del uso de IA

Formato del Anexo 1 de la Política de uso de Inteligencia Artificial en actividades académicas 2026-20.

Curso: Teoría de Compiladores (1ACC0218). Ciclo: 2026-20. Evaluación: TB2, Hito 2. Sección: 4889.

Tabla 15
Bitácora del uso de IA

#	Fecha	Herramienta de IA	Qué consultamos	Resultado IA	Qué se cambió	Cómo se validó
1	09/11/2026	Claude Code (Anthropic)	Revisar el Hito 1 contra el enunciado del trabajo y la rúbrica.	Lista de requisitos cumplidos y faltantes; ejecutó main.py con las entradas de prueba.	Aceptamos la lista. Decidimos completar primero el código y dejar el informe para después.	Leímos el enunciado y corrimos main.py con cada entrada.
2	09/11/2026	Claude Code (Anthropic)	Completar el código del Hito 1 con más validaciones y pruebas.	Evaluación de expresiones constantes, validación de tiempos mayores a 0, ocho casos de prueba y README.	Se aceptó.	run_tests.sh y revisión de cada salida esperada.
3	09/11/2026	Claude Code (Anthropic)	Avanzar el Hito 2: semántico completo, back end y pruebas.	Encontró el error de precedencia de la gramática; propuso la gramática E/T/F, 21 verificaciones, el Modelo, un primer simulador y casos con salida esperada.	Se aceptó. El primer simulador se reemplazó después (fila 5).	Calculamos a mano los resultados de cinco simulaciones.
4	09/11/2026	Claude Code (Anthropic)	Primera versión del informe y la presentación; script de derivaciones; repositorio en GitHub.	Documentos generados desde el código, tools/derivaciones.py y repositorio con milestones e issues.	Completamos la carátula. Dejamos el repositorio privado.	El script comprueba que cada producción usada pertenezca a G; revisamos el PDF.
5	09/11/2026	[Nombre de la segunda herramienta]	Revisión crítica del Hito 2.	Señaló tres fallas (recursos compartidos, capacidad casi cero, ámbito global) y entregó una versión con simulador por eventos, opción --ir y 11 regresiones.	Adoptamos la versión corregida.	11 pruebas de regresión.
6	10/11/2026	Claude Code (Anthropic)	Comprobar si las tres fallas eran reales y si la versión corregida funcionaba.	Reprodujo las tres fallas en la versión anterior; comparó ambos simuladores en 3000 modelos aleatorios y en un caso calculado a mano.	Subimos la versión corregida a GitHub (tag hito-2).	Caso a mano: 4 atendidos, espera 1.50, 90 % y 25 %. Clon limpio con todas las pruebas en OK.
7	10/11/2026	Claude Code (Anthropic)	Reescribir la redacción del informe con normas APA 7.	Texto nuevo del informe, tablas y figuras en formato APA y esta bitácora.	[Completar: qué cambió el grupo en el texto]	Cada dato del informe sale de una ejecución del compilador.